

SIKSHA ‘O’ ANUSANDHAN

(Deemed To Be University)

Accredited (3rd Cycle) by NAAC with **A++ Grade**

Admission Batch: 2023 | Session: 2025–26

Major Project Report of

MACHINE LEARNING CONCEPT-2 (CSE 3968)

On

MLC Network Detection System

Network Anomaly Detection Using Machine Learning

IoT-23 Dataset · Isolation Forest · One-Class SVM · Autoencoder · Random Forest · Streamlit
Deployment

Submitted by:

Name	Registration Number
NAME 1	2341_____
NAME 2	2341_____
NAME 3	2341_____
NAME 4	2341_____

Section: 2341_____ Semester: _____ Branch: Computer Science & Engineering

SOA LOGO

Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER), S'O'A
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030

May, 2026

CERTIFICATE

This is to certify that the Major Project Report titled “**MLC Network Detection System: Network Anomaly Detection Using Machine Learning**” submitted by the students listed below has been carried out under the supervision of the undersigned and is a bonafide record of work done in partial fulfilment of the requirements of **Machine Learning Concept-2 (CSE 3968)**, academic session 2025–26.

Name	Registration No.
NAME 1	2341_____
NAME 2	2341_____
NAME 3	2341_____
NAME 4	2341_____

Supervisor / Faculty Guide

Name: _____

Designation: _____

Dept: CSE, ITER, S'O'A

Head of Department

Name: _____

Designation: Professor & HOD

Dept: CSE, ITER, S'O'A

Date: _____**Place:** Bhubaneswar, Odisha

ABSTRACT

The **MLC Network Detection System** is a machine learning-based network anomaly detection framework designed to classify network traffic as *Benign* or *Malicious*. Traditional signature-based Intrusion Detection Systems (IDS) become inadequate against novel and zero-day attack patterns. This project employs supervised and unsupervised machine learning on the **IoT-23 dataset** – a real-world collection of labeled IoT connection logs from CTU/Stratosphere Laboratory.

The complete pipeline covers: dataset loading, cleaning, Exploratory Data Analysis (EDA), feature engineering, preprocessing, model training, evaluation, model persistence using `.pkl`, and Streamlit web app deployment. A sample of **50,000 connection records** was used. Five model categories were trained and compared – Bagging, Boosting, Random Forest, Isolation Forest, One-Class SVM, and a deep learning Autoencoder.

Key Results: Random Forest achieved Accuracy 0.9979, Precision 0.9962, Recall 1.0000, and F1-Score 0.9981. Bagging (Decision Tree) and Gradient Boosting both reached perfect accuracy of 1.0000 on the test set. The Streamlit app provides manual traffic prediction, row-level testing from `testing_data.csv`, and interactive visualizations.

Keywords: Network Anomaly Detection, IoT-23, Random Forest, Isolation Forest, One-Class SVM, Autoencoder, Ensemble Learning, Streamlit, Intrusion Detection, Feature Engineering.

Contents

1	Introduction	7
1.1	Background and Motivation	7
1.2	Machine Learning for Network Security	7
1.3	Why IoT-23 and Not KDD Cup 99?	7
1.4	Project Overview	7
2	Problem Statement	7
3	Objectives	8
4	Literature Review	8
4.1	Signature-Based and Rule-Based Systems	8
4.2	Ensemble Machine Learning Approaches	8
4.3	Anomaly Detection Models	8
4.4	IoT-Specific Datasets and Research Gap	8
5	Dataset Description and Understanding	9
5.1	Dataset: IoT-23	9
5.2	Dataset Columns	9
5.3	Target Variable	9
6	Data Preprocessing	10
6.1	Raw Data Challenges	10
6.2	Loading and Cleaning	10
6.3	Preprocessing Pipeline	10
6.4	Train-Test Split	10
7	Exploratory Data Analysis (EDA)	11
7.1	Missing Value Analysis	11
7.2	Numeric Feature Distributions	11
7.3	Correlation Heatmap	11
7.4	EDA Intuition for Attack Detection	12
8	Feature Engineering	12
8.1	Why Feature Engineering Is Critical	12
8.2	Engineered Features and Security Rationale	13
8.3	Feature Importance	13
9	Model Development and Justification	13
9.1	Model Selection Philosophy	14
9.2	Bagging Models	14
9.3	Boosting Models	14
9.4	Unsupervised Anomaly Detection	14
9.5	Deep Learning: Autoencoder	14
10	System Architecture	14
10.1	Two-Phase Architecture	14
10.2	System Architecture Diagram	15

10.3 Data Flow Diagram	15
10.4 Training Pipeline	15
10.5 Technology Stack	16
11 Evaluation Metrics	16
12 Experimental Results	17
12.1 Bagging and Boosting Model Comparison	17
12.2 Independent Model Comparison	17
12.3 Result Analysis	17
13 Streamlit Web Application	18
13.1 What is Streamlit?	18
13.2 Application Structure	18
13.3 Implemented Features	18
13.4 How to Run	18
14 Applications	19
15 Challenges and Limitations	19
15.1 Technical Challenges	19
15.2 Limitations	20
16 Future Scope	20
17 Conclusion	20
References	21
Appendix: Source Code Listings	22

List of Figures

1	Distribution of Benign and Malicious traffic in the IoT-23 sample (50,000 records).	10
2	Missing value analysis: columns with highest missing data percentages.	11
3	Log-transformed distributions of key numeric features.	11
4	Correlation heatmap of numeric features in the IoT-23 connection dataset.	11
5	Feature importance ranking from the saved Random Forest model.	13
6	Two-phase system architecture: Training Phase (Notebook) and Deployment Phase (Streamlit).	15
7	Data flow from raw IoT-23 connection logs to final Benign/Malicious prediction. .	15
8	Bagging and Boosting model accuracy comparison (Cell 19).	17
9	Confusion matrix for Random Forest – best performing model.	18
10	Streamlit application home page with model status and sidebar.	19
11	Manual traffic prediction form in the Streamlit application.	19
12	testing_data.csv row-level prediction with actual vs. predicted label comparison. .	19

List of Tables

1	Challenges in Traditional Network Intrusion Detection	7
2	IoT-23 Zeek Connection Log – Column Descriptions	9
3	Preprocessing Steps and Justification	10
4	Engineered Features – Description and Security Rationale	13
5	All Models Used – Type, Mechanism, and Justification	14
6	Technology Stack Used in the Project	16
7	Evaluation Metrics – Definitions and Relevance	16
8	Bagging and Boosting Model Comparison (Ranked by Accuracy)	17
9	Independent Model Comparison (Ranked by F1-Score)	17
10	Project Files and Their Purpose	22

1. Introduction

1.1 Background and Motivation

Cybersecurity is one of the most critical domains in modern computing. As Internet-of-Things (IoT) devices, cloud systems, and smart infrastructure grow exponentially, network traffic volume and attack diversity increase in parallel. Attack vectors range from port scanning and unauthorized access to malware communication, botnet coordination, and denial-of-service floods [2, ?]. Traditional Intrusion Detection Systems (IDS) rely on *signature-based detection* – comparing incoming packets against known attack signatures. While effective for catalogued threats, they fail against new attack categories. This limitation, combined with the explosive volume of IoT network logs, creates an urgent need for intelligent, adaptive, data-driven security solutions.

1.2 Machine Learning for Network Security

Machine learning models learn behavioral patterns from historical data and identify deviations from normal behavior – even for previously unseen attacks. This project trains multiple ML models on the **IoT-23 dataset** and evaluates their ability to classify connections as *Benign* or *Malicious* [?, ?].

1.3 Why IoT-23 and Not KDD Cup 99?

Dataset Selection Rationale: The KDD Cup 99 dataset, while historically important, contains approximately 78% duplicate records, causing severely inflated accuracy metrics [?]. Its traffic patterns are based on simulated 1998 DARPA data and do not represent modern network behavior. The **IoT-23 dataset** from Stratosphere Laboratory captures real IoT malware scenarios (Mirai, Torii, Hakai) using Zeek/Bro – making it far more representative of current network anomaly detection challenges [2].

1.4 Project Overview

The **MLC Network Detection System** is a full end-to-end ML project covering: dataset procurement from IoT-23, data cleaning of Zeek logs, EDA, feature engineering, training of five model categories, serialisation of the best model as a .pk1 file, and deployment as an interactive Streamlit application.

2. Problem Statement

Modern networks generate millions of connection records per hour. Manually analyzing these for attacks is impossible. Existing signature-based IDS tools address only a portion of this problem.

Table 1: Challenges in Traditional Network Intrusion Detection

Challenge	Impact
Signature dependency	Cannot detect unknown or zero-day attacks
Frequent update need	High maintenance overhead for security teams
High false positive rate	Alert fatigue; real threats get missed
IoT device proliferation	Millions of low-security endpoints expand attack surface
Encrypted traffic growth	Deep packet inspection becomes limited
Manual log analysis	Impossible at gigabyte-per-second traffic scale
Evolving malware	Polymorphic code bypasses fixed rule sets

Core Problem: Build an ML system that (a) learns network traffic behavior from labeled IoT logs, (b) classifies connections as Benign or Malicious, (c) saves the trained model for reuse, and (d) exposes the model through an interactive Streamlit web interface.

3. Objectives

1. Understand and preprocess real-world IoT network logs from the IoT-23 dataset.
2. Perform comprehensive EDA to identify traffic patterns and class distributions.
3. Engineer meaningful features from raw network fields encoding attack-indicative signals.
4. Train and compare multiple ML models – Bagging, Boosting, Random Forest, Isolation Forest, One-Class SVM, and Autoencoder.
5. Evaluate all models using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.
6. Serialize the best-performing model as a `.pkl` file using Joblib/Pickle.
7. Build an interactive Streamlit application that predicts from manual input or `testing_data.csv` rows.
8. Provide meaningful visualizations of byte/packet volume, port behavior, and prediction confidence.

4. Literature Review

4.1 Signature-Based and Rule-Based Systems

Denning's 1987 paper established the foundational framework for host-based and network-based IDS [?]. Commercial tools (Snort, Suricata) implement continuously updated signature libraries. McHugh [?] noted that KDD Cup 99 dataset evaluations showed inflated scores due to over 78% redundancy, making results unreliable for real-world deployment.

4.2 Ensemble Machine Learning Approaches

Breiman's Random Forest [?] remains widely deployed in cybersecurity; it handles mixed feature types, resists overfitting through bagging, and outputs feature importance scores. Friedman's Gradient Boosting [?] improves detection of difficult minority-class attack samples via iterative residual correction. Freund and Schapire's AdaBoost [?] sequentially re-weights misclassified samples.

4.3 Anomaly Detection Models

Liu et al. introduced Isolation Forest [?] – detecting anomalies by randomly partitioning feature space; outliers are isolated in fewer splits. Schölkopf et al. proposed One-Class SVM [?] to learn a hyperplane separating normal data from the origin in kernel space. Sakurada and Yairi [?] studied Autoencoder-based anomaly detection where high reconstruction error signals an anomaly.

4.4 IoT-Specific Datasets and Research Gap

García et al. released IoT-23 [2] through Stratosphere Laboratory, Czech Technical University – 23 real IoT malware scenarios in Zeek/Bro format. Ring et al. [?] surveyed network-based IDS datasets and highlighted the superiority of real captured traffic over simulated datasets. This project addresses the gap by comparing supervised, unsupervised, and deep learning models on IoT-23, and deploying results via Streamlit.

5. Dataset Description and Understanding

5.1 Dataset: IoT-23

The project uses the **IoT-23 Dataset** [2] captured using Zeek (formerly Bro). The specific scenario file used is:

```
opt/Malware-Project/BigDataset/IoTScenarios/CTU-IoT-Malware-Capture-1-1/bro/conn.log.  
labeled
```

Only **50,000 rows** were loaded (`SAMPLE_SIZE = 50_000`) to ensure practical training time while preserving statistical representativeness. See Code 2 (Appendix) for the full loading implementation.

5.2 Dataset Columns

Table 2: IoT-23 Zeek Connection Log – Column Descriptions

Column	Description
ts	Unix epoch timestamp of connection start
uid	Unique connection identifier (dropped during training)
id.orig_h	Source/origin IP address
id.orig_p	Source/origin port number
id.resp_h	Destination/response IP address
id.resp_p	Destination/response port number
proto	Transport protocol: TCP, UDP, ICMP
service	Application-layer service: HTTP, DNS, SSH, etc.
duration	Elapsed connection time in seconds
orig_bytes	Bytes sent by origin host
resp_bytes	Bytes sent by response host
conn_state	Zeek connection state (S0, SF, REJ, RSTO, etc.)
missed_bytes	Bytes missed during packet capture
history	Sequence of packet-level events
orig_pkts / resp_pkts	Packets sent by origin/response
label	Raw Zeek label: malware family name or Benign tag

5.3 Target Variable

The raw `label` column contains malware family names (e.g., “Malicious-Mirai”) or “Benign”. The function `extract_traffic_type()` (Code 3, Appendix) converts this into a binary target: **Benign** or **Malicious**. This is a binary classification problem with mixed numeric and categorical features.

[IMAGE PLACEHOLDER]

Traffic class distribution – Benign vs. Malicious count plot
 Source: Cell 11 – `sns.countplot(data=df, x="traffic_type")`

Figure 1: Distribution of Benign and Malicious traffic in the IoT-23 sample (50,000 records).

6. Data Preprocessing

6.1 Raw Data Challenges

The IoT-23 Zeek log presents several preprocessing challenges: missing values encoded as `-`, empty strings, or `(empty)`; numeric columns stored as object type due to mixed content; high-cardinality categorical columns (`history`); and identifier columns (`uid`, raw IPs) that should be excluded from model training.

6.2 Loading and Cleaning

The dataset is loaded with explicit `na_values` to handle Zeek's missing value encoding, and all numeric columns are coerced to numeric format. Full code is in Code 2 (Appendix). The target extraction function is in Code 3 (Appendix).

6.3 Preprocessing Pipeline

A Scikit-learn `ColumnTransformer` applies consistent transformations during both training and prediction. See Code ?? (Appendix) for the full pipeline implementation.

Table 3: Preprocessing Steps and Justification

Step	Applied To	Reason
Median Imputation	Numeric columns	Robust to skewed distributions; unaffected by outliers
StandardScaler	Numeric columns	Zero mean, unit variance – required by SVM and distance models
Mode Imputation	Categorical columns	Fills missing values with most common category
OneHotEncoder	Categorical columns	Converts nominal labels to binary indicator features
Train-Test Split 80/20	Full dataset	Stratified to preserve Benign/Malicious ratio

6.4 Train-Test Split

The dataset was split 80/20 with stratification (Code 6, Appendix):

- **Training set:** 40,000 records (80%)
- **Test set:** 10,000 records (20%)

7. Exploratory Data Analysis (EDA)

EDA answers three key questions: (1) What is the Benign/Malicious distribution? (2) How are numeric features distributed? (3) Which features correlate with each other and the target? Code 4 (Appendix) implements all EDA visualizations.

7.1 Missing Value Analysis

The dataset had significant missing values in `service`, `history`, `orig_bytes`, and `resp_bytes` – due to Zeek connection logging behavior where short/incomplete connections may not populate all fields.

[IMAGE PLACEHOLDER]

Missing value percentage per column – bar chart from Cell 10

Figure 2: Missing value analysis: columns with highest missing data percentages.

7.2 Numeric Feature Distributions

Network byte counts and packet counts are highly right-skewed. Log transformation ($\log(1 + x)$) was applied for visualization clarity (Code 4, Appendix).

[IMAGE PLACEHOLDER]

Log-transformed distributions of duration, orig_bytes, resp_bytes – Cell 11

Figure 3: Log-transformed distributions of key numeric features.

7.3 Correlation Heatmap

A Pearson correlation heatmap was generated for numeric columns. Notable findings: `orig_pkts` and `orig_ip_bytes` are strongly correlated; `resp_pkts` and `resp_ip_bytes` show similar correlation; weak correlations between origin and response metrics suggest asymmetric traffic – indicative of scanning and malware C2 patterns.

[IMAGE PLACEHOLDER]

Numeric feature correlation heatmap (coolwarm) – Cell 11

Figure 4: Correlation heatmap of numeric features in the IoT-23 connection dataset.

7.4 EDA Intuition for Attack Detection

Why EDA Matters for Network Anomaly Detection:

- Malicious connections often show *zero response bytes* – target unavailable or one-way scan.
- Port scanning creates many connections with *very short duration* and *low byte count*.
- Botnet malware contacts specific destination ports with *repetitive packet patterns*.
- *Missing service* fields indicate incomplete or refused connections – a strong anomaly signal.
- Asymmetric `orig_bytes/resp_bytes` ratios indicate data exfiltration or C2 communication.

8. Feature Engineering

8.1 Why Feature Engineering Is Critical

Raw network log fields capture what happened; engineered features capture *how* it happened – the behavioral signature distinguishing attacks from legitimate traffic. A model receiving raw `orig_bytes` and `resp_bytes` sees two numbers; receiving `orig_resp_byte_ratio` it sees whether communication was one-sided. This transformation is what separates weak models from strong ones on network data [?]. Full implementation is in Code 5 (Appendix).

8.2 Engineered Features and Security Rationale

Table 4: Engineered Features – Description and Security Rationale

Feature	How Created	Security Relevance
service_missing	isna() flag	Missing service often indicates failed/refused connection
history_missing	isna() flag	Incomplete packet history suggests truncated traffic
orig_bytes_missing	isna() flag	No bytes sent may indicate scan or one-way probe
resp_bytes_missing	isna() flag	No response bytes – target unreachable or filtering
hour	Timestamp extraction	Malware often activates at unusual hours
dayofweek	Timestamp extraction	Periodic botnet check-ins follow day-of-week patterns
total_bytes	orig + resp bytes	Scanning produces very low totals
total_pkts	orig + resp packets	Packet volume distinguishes high-frequency attacks
bytes_per_pkt	total_bytes/total_pkt	Malware C2 often uses small fixed-size packets
orig_resp_byte_ratio	orig/resp bytes	High ratio = exfiltration; low ratio = DDoS target
orig_resp_pkt_ratio	orig/resp packets	Asymmetric traffic pattern indicator
orig_port_is_well_known	port $\leq$ 1024	Source using privileged port unusual in IoT
resp_port_is_well_known	port $\leq$ 1024	Malware targets ports 22, 23, 80, 443
orig_ip_prefix	First two IP octets	Groups traffic by source subnet
resp_ip_prefix	First two IP octets	Groups traffic by destination subnet

8.3 Feature Importance

After training Random Forest, feature importance scores were extracted. Features related to port behavior, connection state, packet count, byte volume, and missing-value flags ranked highest.



Figure 5: Feature importance ranking from the saved Random Forest model.

9. Model Development and Justification

9.1 Model Selection Philosophy

This project trains both *supervised* and *unsupervised* models. Supervised models (Random Forest, Bagging, Boosting) use labeled data and achieve high accuracy on known attack patterns. Unsupervised anomaly detection models (Isolation Forest, One-Class SVM, Autoencoder) learn only from normal traffic and can detect unknown attacks. Code 7 (Appendix) implements the Bagging/Boosting training. Code 9 (Appendix) implements the anomaly detection models.

Table 5: All Models Used – Type, Mechanism, and Justification

Model	Category	Why Selected
Bagging + Decision Tree	Supervised	Reduces variance through bootstrap aggregation
Random Forest	Supervised	State-of-the-art ensemble; handles mixed features; robust
AdaBoost	Supervised	Focuses iteratively on misclassified samples
Gradient Boosting	Supervised	High performance via sequential residual correction
Hist Gradient Boosting	Supervised	Memory-efficient GBM for large datasets
Isolation Forest	Unsupervised	Detects outliers without labels; sparse data efficient
One-Class SVM	Unsupervised	Learns boundary of normal traffic in kernel space
Autoencoder (DL)	Deep Learning	Reconstructs normal traffic; flags high-error inputs

9.2 Bagging Models

Bagging with Decision Tree: Multiple trees trained on bootstrap samples; final prediction by majority vote. Reduces variance compared to a single tree. **Random Forest:** Extends bagging by additionally randomizing feature selection at each split – best-in-class for tabular data [?].

9.3 Boosting Models

AdaBoost [?]: Sequentially re-weights misclassified samples. **Gradient Boosting** [?]: Each tree fits the negative gradient of the loss function – extremely powerful for structured data. **Hist-GradientBoosting:** Bins continuous features into histograms for significantly faster training on 50,000+ records.

9.4 Unsupervised Anomaly Detection

Isolation Forest [?]: Randomly selects a feature and split point. Anomalies are isolated in fewer splits because they are rare and different. **One-Class SVM** [?]: Finds a hyperplane in kernel space separating normal data from the origin – test points beyond this boundary are flagged as anomalies.

9.5 Deep Learning: Autoencoder

Architecture: Input → Dense(64, relu) → Dense(32, relu) → Dense(16, relu) → Dense(32, relu) → Dense(64, relu) → Output. Trained on *benign traffic only* using MSE loss with EarlyStopping [?]. Reconstruction error threshold at the 95th percentile separates benign from malicious at inference time.

10. System Architecture

10.1 Two-Phase Architecture

The system operates in two clearly separated phases:

- **Training Phase:** Dataset is loaded, cleaned, engineered, split, and used to train all models.

The best model is saved as a `.pkl` file. This phase runs once in Jupyter Notebook.

- **Deployment Phase:** Streamlit app loads the `.pkl` pipeline. No retraining occurs. The saved preprocessing and model are applied to new input in real time.

10.2 System Architecture Diagram

[IMAGE: SYSTEM ARCHITECTURE]

Block Diagram: Dataset → Cleaning → Feature Engineering →

Training → Best Model → `.pkl` → Streamlit App → Prediction

Draw in draw.io and insert here as: `\includegraphics[width=\linewidth]{system_arch.png}`

Figure 6: Two-phase system architecture: Training Phase (Notebook) and Deployment Phase (Streamlit).

10.3 Data Flow Diagram

[IMAGE: DATA FLOW DIAGRAM]

Data Flow: IoT-23 Raw Logs → Zeek Parser → Pandas DataFrame

→ Feature Engineering → ColumnTransformer → Model → Prediction

Draw in draw.io and insert here as: `\includegraphics[width=\linewidth]{data_flow.png}`

Figure 7: Data flow from raw IoT-23 connection logs to final Benign/Malicious prediction.

10.4 Training Pipeline

The complete pipeline (Code 8, Appendix) follows these steps:

1. Load IoT-23 dataset (50,000 rows, tab-separated)
2. Clean missing/invalid values (-, empty, (empty))
3. Extract binary target (Benign / Malicious)
4. Perform EDA and visualizations
5. Apply feature engineering (15 new features)
6. Split: 80% train / 20% test (stratified)
7. Apply ColumnTransformer preprocessing pipeline
8. Train all models (Bagging, Boosting, Anomaly, Autoencoder)
9. Evaluate using Accuracy, Precision, Recall, F1-score, Confusion Matrix
10. Select best model and serialize as .pkl / .joblib
11. Deploy via Streamlit (app.py)

10.5 Technology Stack

Table 6: Technology Stack Used in the Project

Category	Technology / Library
Programming Language	Python 3.10+
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
ML Framework	Scikit-learn (Pipeline, ColumnTransformer, Ensemble, SVM)
Deep Learning	TensorFlow 2.x / Keras (Autoencoder)
Model Persistence	Joblib, Pickle
Web Framework	Streamlit
Development Environment	Jupyter Notebook / Google Colab
Dataset Source	IoT-23 (Stratosphere Laboratory, CTU)

11. Evaluation Metrics

In network anomaly detection, accuracy alone is insufficient. A model labeling everything as Benign achieves high accuracy on an imbalanced dataset while being useless. The following metrics were computed for every model:

Table 7: Evaluation Metrics – Definitions and Relevance

Metric	Formula	Importance in IDS
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall correctness; misleading on imbalanced data
Precision	$\frac{TP}{TP+FP}$	Reduces false alarms; important for analyst workload
Recall	$\frac{TP}{TP+FN}$	Detects real attacks; critical in security context
F1-Score	$\frac{2 \cdot P \cdot R}{P+R}$	Balances precision and recall
Confusion Matrix	2×2 matrix	Shows distribution of TP, TN, FP, FN errors

Why Recall is Most Critical: In cybersecurity, a False Negative (missing a real attack) is far more dangerous than a False Positive (flagging a benign connection). A system with

Recall = 1.0 detects every malicious connection, even if it occasionally generates false alerts. This is the dominant objective in IDS design.

12. Experimental Results

12.1 Bagging and Boosting Model Comparison

All ensemble models were trained with the full preprocessing pipeline (Code 7, Appendix). Results from the 20% test set:

Table 8: Bagging and Boosting Model Comparison (Ranked by Accuracy)

#	Model	Accuracy	Precision	Recall	F1-Score
1	Bagging – Decision Tree	1.0000	1.00	1.00	1.00
2	Boosting – Gradient Boosting	1.0000	1.00	1.00	1.00
3	Boosting – Hist Gradient Boost	0.9987	1.00	1.00	1.00
4	Bagging – Random Forest	0.9979	1.00	1.00	1.00
5	Boosting – AdaBoost	0.9920	0.99	0.99	0.99

[IMAGE PLACEHOLDER]

Bagging/Boosting accuracy bar chart – Cell 19: `sns.barplot(data=results_df, y="model", x="accuracy")`

Figure 8: Bagging and Boosting model accuracy comparison (Cell 19).

12.2 Independent Model Comparison

The four independent models were evaluated using the trained preprocessor (Code 9, Appendix):

Table 9: Independent Model Comparison (Ranked by F1-Score)

#	Model	Accuracy	Precision	Recall	F1-Score
1	Random Forest	0.9979	0.9962	1.0000	0.9981
2	Isolation Forest	0.7797	0.7168	1.0000	0.8350
3	One-Class SVM	0.7771	0.7144	1.0000	0.8334
4	Autoencoder	0.8158	0.9462	0.7100	0.8113

12.3 Result Analysis

- **Random Forest** achieved the best balance across all metrics. Perfect Recall (1.0000) and near-perfect Precision (0.9962) make it ideal for production IDS use.
- **Isolation Forest** and **One-Class SVM** achieved perfect Recall – detected every malicious connection. However, lower Precision (~ 0.71) indicates they also flagged some benign traffic.
- **Autoencoder** showed the highest Precision after Random Forest (0.9462), but Recall of 0.71 means it missed $\sim 29\%$ of real attacks – due to threshold sensitivity in reconstruction error.

[IMAGE PLACEHOLDER]

Confusion matrix for Random Forest (best model) – Cell 19: `sns.heatmap(cm, annot=True)`

Figure 9: Confusion matrix for Random Forest – best performing model.

Best Model: Random Forest

Accuracy	Precision	Recall	F1-Score
0.9979	0.9962	1.0000	0.9981

Saved as: `best_network_anomaly_model.pkl`

13. Streamlit Web Application

13.1 What is Streamlit?

Streamlit is an open-source Python framework for building interactive data science and ML web applications [?]. No HTML, CSS, or JavaScript is required. In this project, Streamlit turns the trained `.pkl` model into a usable web tool for traffic prediction and visualization.

13.2 Application Structure

Main file: `app.py` **Loaded model:** `best_network_anomaly_model.pkl` **Prediction classes:** Benign / Malicious

13.3 Implemented Features

- **Model Loading:** App loads the saved pipeline using `@st.cache_resource` with Joblib. Sidebar shows Model Loaded / error on version mismatch.
- **Manual Prediction Form:** Form with fields: Origin IP, Port, Response IP, Port, Protocol, Service, Duration, Bytes, Connection State, Packets, History. Clicking **Predict Traffic** applies feature engineering and sends to the loaded pipeline.
- **testing_data.csv Row Prediction:** Selects a row by index – displays actual label and predicted label for direct comparison.
- **Feature Engineering in App:** Same `engineer_features()` function from the notebook is applied to ensure matching feature format used during training.
- **Prediction Output:** Clear result (Benign / Malicious) with probability confidence score.
- **Visualizations:** Byte Volume chart, Packet Volume chart, Port Summary, and Prediction Confidence bar chart.
- **Sidebar:** Model status, Project Info, Label Guide.

13.4 How to Run

```
cd D:\MLC_Projects
python -m streamlit run app.py
# Then open: http://localhost:8501
```

[IMAGE PLACEHOLDER]

Streamlit home page – title, sidebar model status, label guide screenshot

Figure 10: Streamlit application home page with model status and sidebar.

[IMAGE PLACEHOLDER]

Manual prediction form – user enters traffic fields, clicks Predict Traffic

Figure 11: Manual traffic prediction form in the Streamlit application.

[IMAGE PLACEHOLDER]

testing_data.csv row selector – actual label vs predicted label comparison

Figure 12: testing_data.csv row-level prediction with actual vs. predicted label comparison.

14. Applications

- **Enterprise Network Monitoring:** Deployed at network edge to classify live Zeek-captured connections.
- **IoT Security:** Protecting smart home, industrial IoT, and healthcare IoT from malware.
- **Cloud Infrastructure:** Monitoring east-west traffic inside cloud VPCs for lateral movement detection.
- **Smart Cities:** Protecting traffic management, utilities, and surveillance infrastructure.
- **Cybersecurity Education:** Demonstrating a complete ML pipeline from data to deployment.
- **Preliminary IDS:** First-pass filtering to reduce false alerts for human security analysts.

15. Challenges and Limitations

15.1 Technical Challenges

- **Mixed data types:** Zeek logs use – for missing values, requiring explicit `na_values` during loading.

- **Binary label format for anomaly models:** Isolation Forest and One-Class SVM output +1/-1; Autoencoder outputs reconstruction error. These required careful threshold conversion to match the Benign/Malicious labels.
- **Autoencoder threshold selection:** The reconstruction error percentile threshold directly impacts Precision-Recall trade-off.
- **Scikit-learn version dependency:** The .pkl pipeline requires the same Scikit-learn version used during training; version mismatches cause Streamlit loading errors.
- **One-Class SVM training time:** $O(n^2)$ complexity required subsampling to 10,000 benign records.

15.2 Limitations

- Only 50,000 of the available millions of IoT-23 records were used.
- The model is trained on IoT-23 Scenario 1; generalization to other malware families may require retraining.
- Real-time packet capture is not implemented; the app works on pre-recorded connection logs.
- The current system predicts one record at a time; batch CSV upload is not available.

16. Future Scope

- **Real-time packet capture:** Integrate with Zeek or Scapy to capture live traffic.
- **Batch CSV prediction:** Add file upload in Streamlit to predict a full CSV of records.
- **Cloud deployment:** Host the Streamlit app on AWS, Azure, or Streamlit Cloud.
- **SIEM integration:** Export predictions to a Security Information and Event Management system.
- **Explainable AI:** Integrate SHAP or LIME to explain why a connection was flagged malicious.
- **All IoT-23 scenarios:** Train on all 23 malware scenarios for broader coverage.
- **Federated learning:** Train across distributed network nodes without centralizing raw logs.
- **Edge AI:** Deploy a lightweight model on IoT gateway devices for on-device detection.

17. Conclusion

The **MLC Network Detection System** successfully developed and deployed a complete machine learning framework for network anomaly detection using the real-world IoT-23 dataset. The project covers the full ML lifecycle: raw Zeek log ingestion and cleaning, exploratory analysis, meaningful feature engineering, training of eight diverse models spanning supervised, unsupervised, and deep learning paradigms.

Among all models evaluated, **Random Forest** achieved the best independent performance – Accuracy 0.9979, Precision 0.9962, Recall 1.0000, and F1-Score 0.9981. Its perfect recall means no malicious connection was missed in testing. The best model was serialized as `best_network_anomaly_model.pkl` and integrated into an interactive Streamlit web application providing manual prediction and `testing_data.csv` row-level testing with visualizations.

This project demonstrates that machine learning, applied to well-engineered features from real

network traffic data, can deliver near-perfect anomaly detection accuracy while remaining practical and deployable through modern Python frameworks.

References

- [1] S. García, M. Parmisano, and M. J. Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Zenodo, 2020. <https://doi.org/10.5281/zenodo.4743746>
- [2]
- [3] S. García et al., “IoT-23 Dataset,” Stratosphere Laboratory, CTU, Prague, 2020.
- [4] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. 2nd IEEE Symp. Comput. Intell. Security Def. Appl.*, 2009, pp. 1–6.
- [5] D. E. Denning, “An intrusion-detection model,” *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.
- [6] J. McHugh, “Testing intrusion detection systems: A critique of the 1998 and 1999 DARPA evaluations,” *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 4, pp. 262–294, Nov. 2000.
- [7] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [8] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [9] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [11] B. Schölkopf et al., “Estimating the support of a high-dimensional distribution,” *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [12] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proc. MLSDA Workshop ACM SIGKDD*, 2014.
- [13] M. Ring et al., “A survey of network-based intrusion detection data sets,” *Comput. Secur.*, vol. 86, pp. 147–167, Sep. 2019.
- [14] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security Privacy (SP)*, 2010.
- [15] Streamlit Inc., “Streamlit documentation,” 2024. <https://docs.streamlit.io/>
- [16] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. <https://scikit-learn.org/>

Appendix: Source Code Listings

All code is from `MLC_NetWordDetection_System.ipynb`. Code is referenced in-text by label (e.g., Code 2).

Table 10: Project Files and Their Purpose

File	Purpose
<code>MLC_NetWordDetection_System.ipynb</code>	Jupyter Notebook – complete training pipeline
<code>best_network_anomaly_model.pkl</code>	Serialized best model (Random Forest pipeline)
<code>best_iot23_ensemble_model.joblib</code>	Joblib format of saved model
<code>model_results.csv</code>	Model comparison metrics CSV
<code>app.py</code>	Streamlit web application
<code>testing_data.csv</code>	Test records for Streamlit row-prediction mode
<code>requirements.txt</code>	Python package dependencies

```
pandas>=1.5.0 numpy>=1.23.0 scikit-learn>=1.2.0 tensorflow>=2.10.0
matplotlib>=3.6.0 seaborn>=0.12.0 plotly>=5.10.0 streamlit>=1.20.0 joblib>=1.2.0
```

Code 1: Requirements (`requirements.txt`)

```
1 import os, pickle, warnings
2 from pathlib import Path
3 import joblib, numpy as np, pandas as pd
4 import matplotlib.pyplot as plt, seaborn as sns
5 from sklearn.compose import ColumnTransformer
6 from sklearn.ensemble import (AdaBoostClassifier, BaggingClassifier,
7     GradientBoostingClassifier, HistGradientBoostingClassifier, RandomForestClassifier)
8 from sklearn.impute import SimpleImputer
9 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
10 from sklearn.model_selection import train_test_split
11 from sklearn.pipeline import Pipeline
12 from sklearn.preprocessing import OneHotEncoder, StandardScaler
13 from sklearn.tree import DecisionTreeClassifier
14 warnings.filterwarnings("ignore")
15 sns.set_theme(style="whitegrid")
16
17 RANDOM_STATE = 42
18 SAMPLE_SIZE = 50_000
19
20 file_path = ("opt/Malware-Project/BigDataset/IoTScenarios/"
21             "CTU-IoT-Malware-Capture-1-1/bro/conn.log.labeled")
22
23 columns = [
24     "ts", "uid", "id.orig_h", "id.orig_p", "id.resp_h", "id.resp_p",
25     "proto", "service", "duration", "orig_bytes", "resp_bytes",
26     "conn_state", "local_orig", "local_resp", "missed_bytes", "history",
27     "orig_pkts", "orig_ip_bytes", "resp_pkts", "resp_ip_bytes", "label"
28 ]
29
30 df = pd.read_csv(
31     file_path, sep="\t", skiprows=8, names=columns,
32     nrows=SAMPLE_SIZE, na_values=["-", "", "(empty)"], low_memory=False,
33 )
34 print(f"Loaded shape: {df.shape}")
```

Code 2: Data Loading and Initial Setup (Notebook Cell 6)

```
1 def extract_traffic_type(value):
2     value = str(value)
3     if "Benign" in value:
4         return "Benign"
```

```

5     if "Malicious" in value:
6         return "Malicious"
7     return "Unknown"
8
9 df["traffic_type"] = df["label"].apply(extract_traffic_type)
10 df = df[df["traffic_type"].isin(["Benign", "Malicious"])]
11
12 numeric_cols = [
13     "ts", "id.orig_p", "id.resp_p", "duration", "orig_bytes", "resp_bytes",
14     "missed_bytes", "orig_pkts", "orig_ip_bytes", "resp_pkts", "resp_ip_bytes"
15 ]
16 for col in numeric_cols:
17     df[col] = pd.to_numeric(df[col], errors="coerce")
18
19 print(f"Cleaned shape: {df.shape}")
20 print(df["traffic_type"].value_counts())

```

Code 3: Target Variable Extraction and Numeric Coercion (Notebook Cell 8)

```

1 # Missing value analysis
2 missing_summary = (
3     df.isna().sum()
4     .to_frame("missing_count")
5     .assign(missing_percent=lambda x: 100 * x["missing_count"] / len(df))
6     .sort_values("missing_count", ascending=False)
7 )
8 display(missing_summary.head(15))
9
10 # Class distribution plot
11 plt.figure(figsize=(6, 4))
12 sns.countplot(data=df, x="traffic_type",
13               order=df["traffic_type"].value_counts().index)
14 plt.title("Traffic Type Distribution")
15 plt.show()
16
17 # Log-transformed numeric distributions
18 fig, axes = plt.subplots(1, 3, figsize=(16, 4))
19 for ax, col in zip(axes, ["duration", "orig_bytes", "resp_bytes"]):
20     sns.histplot(np.log1p(df[col].fillna(0)), bins=40, ax=ax)
21     ax.set_title(f"log1p({col})")
22 plt.tight_layout()
23 plt.show()
24
25 # Correlation heatmap
26 plt.figure(figsize=(12, 8))
27 sns.heatmap(df[numeric_cols].corr(numeric_only=True), cmap="coolwarm", center=0)
28 plt.title("Numeric Feature Correlation")
29 plt.show()

```

Code 4: Exploratory Data Analysis Visualizations (Notebook Cell 11)

```

1 def engineer_features(data):
2     data = data.copy()
3
4     # Missing value indicator flags -- strong anomaly signals in Zeek logs
5     for col in ["service", "history", "orig_bytes", "resp_bytes"]:
6         data[f"{col}_missing"] = data[col].isna().astype(int)
7
8     # Time-based features from Zeek epoch timestamp
9     ts = pd.to_datetime(data["ts"], unit="s", errors="coerce")
10    data["hour"] = ts.dt.hour
11    data["dayofweek"] = ts.dt.dayofweek
12
13    # Connection volume aggregate features
14    data["total_bytes"] = data["orig_bytes"].fillna(0) + data["resp_bytes"].fillna(0)
15    data["total_pkts"] = data["orig_pkts"].fillna(0) + data["resp_pkts"].fillna(0)
16    data["bytes_per_pkt"] = data["total_bytes"] / data["total_pkts"].replace(0, np.nan)

```

```

17
18 # Communication asymmetry ratios -- key for detecting malware C2 and exfiltration
19 data["orig_resp_byte_ratio"] = (data["orig_bytes"]
20                                / data["resp_bytes"].replace(0, np.nan))
21 data["orig_resp_pkt_ratio"] = (data["orig_pkts"]
22                                / data["resp_pkts"].replace(0, np.nan))
23
24 # Port behavior features
25 data["orig_port_is_well_known"] = (data["id.orig_p"] <= 1024).astype(int)
26 data["resp_port_is_well_known"] = (data["id.resp_p"] <= 1024).astype(int)
27
28 # Lightweight IP prefix features (avoid one-hot encoding full IPs)
29 data["orig_ip_prefix"] = (data["id.orig_h"].astype(str)
30                            .str.extract(r"^(\\d+\\.\\d+)", expand=False))
31 data["resp_ip_prefix"] = (data["id.resp_h"].astype(str)
32                            .str.extract(r"^(\\d+\\.\\d+)", expand=False))
33
34 # Drop identifiers and raw target fields
35 return data.drop(
36     columns=["uid", "label", "id.orig_h", "id.resp_h", "ts"], errors="ignore")
37
38 model_df = engineer_features(df)
39 X = model_df.drop(columns=["traffic_type"])
40 y = model_df["traffic_type"]
41
42 numeric_features = X.select_dtypes(include=["number", "bool"]).columns.tolist()
43 categorical_features = X.select_dtypes(include=["object", "category"]).columns.tolist()
44 print(f"Rows: {len(model_df)} | Numeric: {len(numeric_features)} | Cat: {len(
    categorical_features)}")

```

Code 5: Feature Engineering Function (Notebook Cell 14)

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y,
3 )
4 # Train shape: (40000, n_features) | Test shape: (10000, n_features)
5
6 try:
7     one_hot = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
8 except TypeError:
9     one_hot = OneHotEncoder(handle_unknown="ignore", sparse=False)
10
11 preprocessor = ColumnTransformer(
12     transformers=[
13         ("num", Pipeline(steps=[
14             ("imputer", SimpleImputer(strategy="median")),
15             ("scaler", StandardScaler()),
16         ]), numeric_features),
17         ("cat", Pipeline(steps=[
18             ("imputer", SimpleImputer(strategy="most_frequent")),
19             ("onehot", one_hot),
20         ]), categorical_features),
21     ],
22     remainder="drop",
23 )
24 print("Train shape:", X_train.shape, "| Test shape:", X_test.shape)

```

Code 6: Train-Test Split and Preprocessing Pipeline (Notebook Cell 16)

```

1 models = {
2     "Bagging - Decision Tree": BaggingClassifier(
3         estimator=DecisionTreeClassifier(random_state=RANDOM_STATE),
4         n_estimators=50, max_samples=0.80, n_jobs=-1,
5         random_state=RANDOM_STATE),
6     "Bagging - Random Forest": RandomForestClassifier(
7         n_estimators=150, max_depth=None, class_weight="balanced",
8         n_jobs=-1, random_state=RANDOM_STATE),

```



```

9     "Boosting - AdaBoost": AdaBoostClassifier(
10         n_estimators=100, learning_rate=0.5, random_state=RANDOM_STATE),
11     "Boosting - Gradient Boosting": GradientBoostingClassifier(
12         n_estimators=120, learning_rate=0.08, max_depth=3,
13         random_state=RANDOM_STATE),
14     "Boosting - Hist Gradient Boosting": HistGradientBoostingClassifier(
15         max_iter=120, learning_rate=0.08, random_state=RANDOM_STATE),
16 }
17
18 trained_models, results = {}, []
19 for name, estimator in models.items():
20     print(f"\nTraining: {name}")
21     pipeline = Pipeline(steps=[
22         ("preprocessor", preprocessor),
23         ("model", estimator),
24     ])
25     pipeline.fit(X_train, y_train)
26     y_pred = pipeline.predict(X_test)
27     accuracy = accuracy_score(y_test, y_pred)
28     trained_models[name] = pipeline
29     results.append({"model": name, "accuracy": accuracy})
30     print(f"Accuracy: {accuracy:.4f}")
31     print(classification_report(y_test, y_pred))
32
33 results_df = (pd.DataFrame(results)
34               .sort_values("accuracy", ascending=False)
35               .reset_index(drop=True))
36 display(results_df)

```

Code 7: Bagging and Boosting Model Training (Notebook Cell 18)

```

1 plt.figure(figsize=(10, 5))
2 sns.barplot(data=results_df, y="model", x="accuracy", palette="viridis")
3 plt.xlim(0, 1)
4 plt.title("Model Accuracy Comparison")
5 plt.show()
6
7 best_model_name = results_df.loc[0, "model"]
8 best_model      = trained_models[best_model_name]
9 best_pred       = best_model.predict(X_test)
10
11 cm = confusion_matrix(y_test, best_pred, labels=["Benign", "Malicious"])
12 plt.figure(figsize=(5, 4))
13 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
14             xticklabels=["Benign", "Malicious"],
15             yticklabels=["Benign", "Malicious"])
16 plt.title(f"Confusion Matrix: {best_model_name}")
17 plt.xlabel("Predicted"); plt.ylabel("Actual")
18 plt.show()
19 print(f"Best model: {best_model_name}")

```

Code 8: Confusion Matrix Visualization (Notebook Cell 19)

```

1 from sklearn.ensemble import IsolationForest, RandomForestClassifier
2 from sklearn.svm import OneClassSVM
3 from sklearn.metrics import (accuracy_score, precision_score,
4                             recall_score, f1_score, classification_report, confusion_matrix)
5 from tensorflow.keras.models import Model
6 from tensorflow.keras.layers import Input, Dense
7 from tensorflow.keras.callbacks import EarlyStopping
8
9 y_train_bin = y_train.map({"Benign": 0, "Malicious": 1}).astype(int)
10 y_test_bin  = y_test.map({"Benign": 0, "Malicious": 1}).astype(int)
11
12 X_train_processed = preprocessor.fit_transform(X_train)
13 X_test_processed  = preprocessor.transform(X_test)
14 if hasattr(X_train_processed, "toarray"):

```

```

15     X_train_processed = X_train_processed.toarray()
16     X_test_processed = X_test_processed.toarray()
17     X_train_processed = X_train_processed.astype("float32")
18     X_test_processed = X_test_processed.astype("float32")
19     X_train_benign = X_train_processed[y_train_bin.values == 0]
20
21 results, trained_models = [], {}
22
23 def evaluate_model(model_name, y_true, y_pred):
24     accuracy = accuracy_score(y_true, y_pred)
25     precision = precision_score(y_true, y_pred, zero_division=0)
26     recall = recall_score(y_true, y_pred, zero_division=0)
27     f1 = f1_score(y_true, y_pred, zero_division=0)
28     results.append({"model": model_name, "accuracy": accuracy,
29                   "precision": precision, "recall": recall, "f1_score": f1})
30     print(f"{model_name} | Acc:{accuracy:.4f} Prec:{precision:.4f}"
31           f" Recall:{recall:.4f} F1:{f1:.4f}")
32     print(classification_report(y_true, y_pred,
33                               target_names=["Benign", "Malicious"], zero_division=0))
34
35 # --- 1. Isolation Forest ---
36 contamination_rate = max(0.01, min(0.49, y_train_bin.mean()))
37 isolation_forest = IsolationForest(
38     n_estimators=150, contamination=contamination_rate,
39     random_state=RANDOM_STATE, n_jobs=-1)
40 isolation_forest.fit(X_train_benign)
41 iso_pred = np.where(isolation_forest.predict(X_test_processed) == -1, 1, 0)
42 trained_models["Isolation Forest"] = isolation_forest
43 evaluate_model("Isolation Forest", y_test_bin, iso_pred)
44
45 # --- 2. One-Class SVM (sampled for speed) ---
46 max_svm_samples = 10000
47 rng = np.random.default_rng(RANDOM_STATE)
48 X_svm_train = X_train_benign[
49     rng.choice(len(X_train_benign), size=min(max_svm_samples,
50     len(X_train_benign)), replace=False)]
51 one_class_svm = OneClassSVM(kernel="rbf", gamma="scale", nu=contamination_rate)
52 one_class_svm.fit(X_svm_train)
53 svm_pred = np.where(one_class_svm.predict(X_test_processed) == -1, 1, 0)
54 trained_models["One-Class SVM"] = one_class_svm
55 evaluate_model("One-Class SVM", y_test_bin, svm_pred)
56
57 # --- 3. Autoencoder ---
58 input_dim = X_train_processed.shape[1]
59 inp = Input(shape=(input_dim,))
60 enc = Dense(64, activation="relu")(inp)
61 enc = Dense(32, activation="relu")(enc)
62 enc = Dense(16, activation="relu")(enc)
63 dec = Dense(32, activation="relu")(enc)
64 dec = Dense(64, activation="relu")(dec)
65 out = Dense(input_dim, activation="linear")(dec)
66 autoencoder = Model(inputs=inp, outputs=out)
67 autoencoder.compile(optimizer="adam", loss="mse")
68 early_stop = EarlyStopping(monitor="val_loss", patience=3,
69                             restore_best_weights=True)
70 autoencoder.fit(X_train_benign, X_train_benign, epochs=20, batch_size=256,
71                 validation_split=0.1, callbacks=[early_stop], verbose=1)
72 train_mse = np.mean(np.square(X_train_benign -
73                               autoencoder.predict(X_train_benign, verbose=0)), axis=1)
74 threshold = np.percentile(train_mse, 95)
75 test_mse = np.mean(np.square(X_test_processed -
76                               autoencoder.predict(X_test_processed, verbose=0)), axis=1)
77 ae_pred = (test_mse > threshold).astype(int)
78 trained_models["Autoencoder"] = autoencoder
79 evaluate_model("Autoencoder", y_test_bin, ae_pred)
80

```

```

81 # --- 4. Random Forest (Supervised) ---
82 random_forest = Pipeline(steps=[
83     ("preprocessor", preprocessor),
84     ("model", RandomForestClassifier(
85         n_estimators=150, class_weight="balanced",
86         random_state=RANDOM_STATE, n_jobs=-1))]
87 random_forest.fit(X_train, y_train_bin)
88 rf_pred = random_forest.predict(X_test)
89 trained_models["Random Forest"] = random_forest
90 evaluate_model("Random Forest", y_test_bin, rf_pred)
91
92 comparison_df = (pd.DataFrame(results)
93                 .sort_values("f1_score", ascending=False)
94                 .reset_index(drop=True))
95 display(comparison_df)

```

Code 9: Anomaly Detection Models: Isolation Forest, One-Class SVM, Autoencoder, Random Forest (Notebook Cell 20)

```

1 # Save best model from independent evaluation
2 best_model_name = comparison_df.loc[0, "model"]
3 best_model      = trained_models[best_model_name]
4
5 if best_model_name == "Autoencoder":
6     best_model.save("best_network_anomaly_autoencoder.keras")
7     print("Best model saved as: best_network_anomaly_autoencoder.keras")
8 else:
9     joblib.dump(best_model, "best_network_anomaly_model.pkl")
10    print("Best model saved as: best_network_anomaly_model.pkl")
11    print(f"Best Model: {best_model_name}")
12
13 # Save also in organized model directory
14 model_dir = Path("models")
15 model_dir.mkdir(exist_ok=True)
16 joblib_path = model_dir / "best_iot23_ensemble_model.joblib"
17 pickle_path = model_dir / "best_iot23_ensemble_model.pkl"
18 metadata_path = model_dir / "model_results.csv"
19
20 joblib.dump(best_model, joblib_path)
21 with open(pickle_path, "wb") as f:
22     pickle.dump(best_model, f)
23 results_df.to_csv(metadata_path, index=False)
24 print(f"Saved to: {joblib_path} and {pickle_path}")

```

Code 10: Model Saving to .pkl and .joblib (Notebook Cell 20 & 22)

```

1 # Load the saved model and verify it produces the same results
2 loaded_model = joblib.load(joblib_path)
3 loaded_accuracy = accuracy_score(y_test, loaded_model.predict(X_test))
4 print(f"Reloaded model accuracy: {loaded_accuracy:.4f}")
5
6 # In Streamlit app.py -- model is loaded as:
7 # @st.cache_resource
8 # def load_model():
9 #     return joblib.load("best_network_anomaly_model.pkl")
10 # model = load_model()
11 # prediction = model.predict(engineered_input_features)

```

Code 11: Load Saved Model and Verify Accuracy (Notebook Cell 23)

Screenshot Checklist – Add before final submission:

1. Fig. 1 – Traffic class distribution (Benign vs. Malicious count plot)

2. Fig. 2 – Missing value analysis per column
3. Fig. 3 – Log-transformed distributions of duration, orig_bytes, resp_bytes
4. Fig. 4 – Correlation heatmap of numeric features
5. Fig. 5 – Top-15 feature importance from Random Forest
6. Fig. 8 – Bagging/Boosting accuracy comparison bar chart
7. Fig. 9 – Confusion matrix of Random Forest
8. Fig. 6 – System architecture block diagram
9. Fig. 7 – Data flow diagram
10. Fig. 10 – Streamlit home page screenshot
11. Fig. 11 – Manual prediction form screenshot
12. Fig. 12 – testing_data.csv row prediction screenshot